UNIVERSIDAD DE ANTIOQUIA

Fundamentos de Sistemas de Información

Informe Laboratorio #2

Reglas de negocio con Spring Boot y Drools

Estudiante: Santiago Correa Marulanda

Docente: Diego José Botia

1. Introducción

En el desarrollo de sistemas de información, las decisiones de una organización suelen depender de políticas que cambian con el tiempo. Cuando estas políticas se implementan exclusivamente mediante estructuras condicionales dispersas en el código, su modificación, revisión y validación se vuelven más difíciles. Los motores de reglas de negocio permiten representar dichas condiciones de manera declarativa y separarlas de las responsabilidades propias de la aplicación [1, 2].

El presente laboratorio tiene como propósito aplicar este enfoque mediante Drools, integrado en una aplicación desarrollada con Java 17 y Spring Boot 3.2.5. A diferencia del ejemplo de evaluación crediticia presentado como guía, el ejercicio asignado plantea el dominio de una aerolínea y exige implementar diez reglas relacionadas con categorías de fidelización, equipaje, retrasos, asignación de asientos y atención preferencial [1].

La solución desarrollada expone servicios HTTP que reciben información de pasajeros y vuelos, evalúan las condiciones del negocio y devuelven resultados estructurados. Además de comprobar las condiciones individuales, se atendieron situaciones en las que varias reglas de negocio pueden activarse durante una misma evaluación, como la incompatibilidad entre un ascenso y una restricción de equipaje o la competencia por un asiento disponible. El funcionamiento se contrastó mediante pruebas automatizadas y solicitudes HTTP reales, cuyas evidencias se presentan en el procedimiento

2. Objetivos

2.1 Objetivo general

Desarrollar y validar una aplicación basada en Spring Boot y Drools que permita evaluar de forma declarativa diez reglas de negocio asociadas a la operación de una aerolínea, garantizando respuestas coherentes ante diferentes características de pasajeros, vuelos y disponibilidad de asientos.

2.2 Objetivos específicos

- Configurar un proyecto Java 17 administrado con Maven e integrar el motor de reglas Drools con Spring Boot.
- Definir las diez reglas solicitadas en lenguaje DRL, conservando sus condiciones, umbrales y acciones de negocio.
- Implementar servicios REST para evaluar un pasajero y conjuntos de pasajeros dentro de un vuelo.

- Gestionar de forma consistente las interacciones entre reglas y la asignación de asientos disponibles.
- Diseñar y ejecutar pruebas automatizadas y escenarios HTTP que permitan comprobar resultados, límites y entradas inválidas.
- Documentar el procedimiento, las evidencias de ejecución y el código fuente disponible en GitHub.

3. Marco teórico

3.1 Reglas de negocio y sistemas BRMS

Una regla de negocio expresa una política del dominio mediante condiciones y consecuencias. Por ejemplo, la condición «equipaje de más de 23 kg» produce la consecuencia «pasajero no elegible para ascenso». Un Business Rules Management System (BRMS) reúne mecanismos para definir, organizar y ejecutar este tipo de políticas, separándolas de la lógica de transporte, interfaz o persistencia de una aplicación. Esta separación facilita su revisión funcional, aunque no elimina la necesidad de probar conflictos entre reglas [1, 2].

3.2 Drools, hechos y lenguaje DRL

Drools es un motor de reglas para el ecosistema Java. Su funcionamiento parte de hechos (objetos que describen el estado del problema), patrones de condiciones y un mecanismo de activación de reglas. Los archivos DRL (Drools Rule Language) contienen declaraciones de reglas y separan típicamente una sección when, que identifica las condiciones, de una sección then, que expresa la consecuencia. La evaluación basada en coincidencias permite que varias reglas resulten aplicables a los mismos hechos [1, 2].

Por esta razón, la corrección no se reduce a escribir diez condiciones aisladas: también es necesario diseñar cómo se aplican sus consecuencias cuando coinciden. Las reglas de ascenso, vetos y asignaciones de asientos son ejemplos de interacciones que deben resolverse explícitamente.

3.3 Componentes KIE

La API KIE (Knowledge Is Everything) proporciona componentes para construir y ejecutar bases de conocimiento. En términos generales, KieContainer representa el acceso a los recursos de reglas, mientras que KieSession permite insertar hechos y ejecutar las reglas correspondientes. Durante la configuración resulta fundamental detectar errores de construcción de los recursos DRL, ya que una definición inválida impediría tomar decisiones correctas [2].

3.4 Spring Boot y API REST

Spring Boot facilita la creación de aplicaciones Java ejecutables y servicios web basados en Spring. En este laboratorio se utilizó para organizar la aplicación y recibir solicitudes HTTP con información de pasajeros y vuelos. La API REST permite desacoplar la entrada y salida de datos de las decisiones tomadas por Drools, utilizando JSON como formato de intercambio [3].

3.5 Maven, pruebas y validación

Apache Maven administra las dependencias y las fases de construcción del proyecto. La ejecución de clean verify permite limpiar artefactos previos y recorrer las fases de compilación, pruebas y empaquetado hasta la verificación del proyecto [4]. Complementariamente, las pruebas automatizadas comprueban condiciones reproducibles, mientras que las peticiones HTTP reales permiten verificar el comportamiento de la aplicación ejecutándose mediante su servidor web.

4. Procedimiento a seguir

4.1 Preparación del proyecto

Se organizó el laboratorio en una carpeta de trabajo local denominada Lab_02_fsi. Allí se creó la estructura estándar de Maven, con pom.xml como archivo principal de configuración, src/main/java para el código fuente, src/main/resources para los recursos de la aplicación y src/test/java para las pruebas. También se añadieron archivos de configuración y documentación. Se utilizaron Java 17, Spring Boot 3.2.5, Drools 7.74.1.Final y Maven como herramientas principales.

El material suministrado por el curso y las evidencias de ejecución se mantuvieron en carpetas locales independientes; estas se excluyeron de Git para evitar publicarlas junto al código fuente.



Ilustración 1 Estructura inicial del proyecto observada desde PowerShell. Fuente: elaboración propia a partir de las evidencias del laboratorio

4.2 Integración y configuración de Drools

En el proyecto se incorporaron las dependencias de Spring Boot, validación, pruebas y Drools. La configuración del motor se centralizó en DroolsConfig.java, responsable de localizar el recurso de reglas, construir la base de conocimiento y comprobar los errores de compilación de KIE. Posteriormente, el archivo DRL se ubicó en una ruta coherente con el paquete que declara: src/main/resources/com/udea/airline/rules/airline-rules.drl.

Esta corrección resolvió una advertencia inicial de Drools relacionada con la diferencia entre la ruta física del recurso y su declaración de paquete, sin modificar los umbrales de las reglas.

```java
@Configuration
public class DroolsConfig {

    @Bean
    public KieContainer kieContainer() {
        KieServices services = KieServices.Factory.get();
        KieFileSystem fileSystem = services.newKieFileSystem();
        fileSystem.write(ResourceFactory.newClassPathResource("com/udea/airline/rules/airline-rules.drl"));
        KieBuilder builder = services.newKieBuilder(fileSystem).buildAll();
        if (builder.getResults().hasMessages(Message.Level.ERROR)) {
            throw new IllegalStateException("Error al compilar las reglas Drools: "
                    + builder.getResults().getMessages(Message.Level.ERROR));
        }
        return services.newKieContainer(builder.getKieModule().getReleaseId());
    }
}
```

Ilustración 2 Configuración del recurso DRL y validación de errores KIE. Fuente: elaboración propia a partir de las evidencias del laboratorio

4.3 Implementación de las diez reglas de negocio

De acuerdo con el enunciado, se implementaron diez reglas en un archivo DRL. La tabla resume las condiciones y acciones exigidas, respetando especialmente la semántica estricta de los comparadores mayor que y menor que [1].

Regla	Condición	Acción
R1. Ascenso	Gold o Platinum; retraso > 60 min	Ascenso a clase ejecutiva.
R2. Prioridad	Edad > 65 años	Check-in prioritario.
R3. Descuento	Equipaje < 10 kg	Descuento del 10 %.
R4. Restricción	Equipaje > 23 kg	No elegible para ascensos.
R5. Emergencia	Edad 18 a 40, preferencia Any y asiento disponible	Asignación y ocupación de salida de emergencia.
R6. Compensación	Retraso > 180 min	Compensación de $200.
R7. Fidelización	Membresía distinta de Basic; vuelo > 5 h	500 puntos adicionales.
R8. Equipaje	Equipaje > 15 kg; vuelo < 2 h	Equipaje no permitido.
R9. Salón VIP	Membresía Platinum	Acceso a salón VIP.
R10. Familias	Viaja con niños y sin preferencia específica	Asiento preferencial familiar.

Para la implementación se asumió un único nivel de fidelización tanto para estatus como para membresía. Se interpretó el intervalo de 18 a 40 años con extremos incluidos, y Any como ausencia de preferencia específica. Estas decisiones responden a ambigüedades del enunciado y fueron documentadas durante el

desarrollo; la compensación se expresa en la unidad monetaria indicada ($), sin atribuirle una divisa que no se especifica [1].

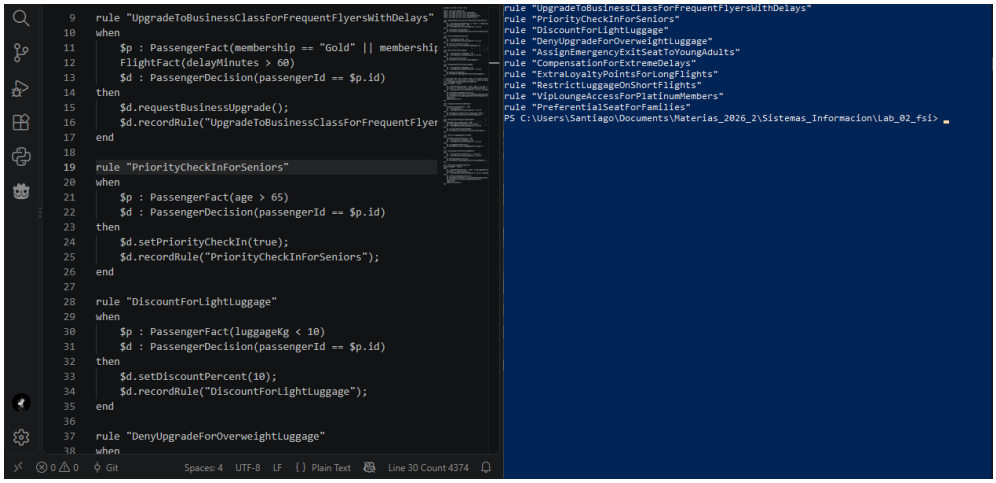


Ilustración 3 Archivo airline-rules.drl y listado de reglas implementadas. Fuente: elaboración propia a partir de las evidencias del laboratorio

4.4 Servicios de evaluación de pasajeros y vuelos

La aplicación proporciona dos operaciones HTTP POST. La evaluación individual recibe los datos de un pasajero junto con el contexto necesario del vuelo; la evaluación de vuelo permite procesar varios pasajeros utilizando un inventario compartido de asientos. El servicio de negocio prepara los datos para Drools y transforma los resultados en respuestas JSON con beneficios, restricciones y asignaciones aplicadas.

Operación	Ruta HTTP	Propósito
Pasajero	POST /api/evaluations/passenger	Calcular beneficios y restricciones de un pasajero.
Vuelo	POST /api/evaluations/flight	Evaluar varios pasajeros y mantener asignaciones consistentes de asientos.

Las entradas inválidas se rechazan mediante respuestas HTTP 400. Las pruebas de integración permitieron comprobar tanto resultados válidos como comportamientos de validación, sin depender exclusivamente de la inspección del código

4.5 Resolución de conflictos entre reglas

Se establecieron criterios para evitar resultados contradictorios. El ascenso a clase ejecutiva queda descartado cuando el pasajero excede 23 kg de equipaje, incluso si por membresía y retraso cumple las condiciones para recibirlo. En la asignación de asientos se prioriza la necesidad familiar cuando coincide con una solicitud de asiento de emergencia y la disponibilidad es insuficiente.

El inventario de asientos se actualiza al producirse la asignación, de manera que un asiento no pueda entregarse a dos pasajeros dentro de la misma evaluación. También se verificó que dos evaluaciones independientes con los mismos datos no acumulen de forma inesperada beneficios como puntos o compensaciones. Estas garantías describen el comportamiento verificado del laboratorio; no constituyen una afirmación de persistencia ni de seguridad transaccional para un sistema aeroportuario en producción.

4.6 Compilación y ejecución de pruebas automatizadas

Se empleó el script scripts/test.ps1 para seleccionar Java 17 en el proceso y ejecutar Maven con clean verify. La primera compilación exitosa reportó 21 pruebas aprobadas: 16 asociadas a BusinessRulesTest y 5 a EvaluationApiTest, sin fallos ni errores. El ejecutable JAR se generó correctamente. Esta etapa permitió confirmar una integración funcional preliminar entre Spring Boot y Drools

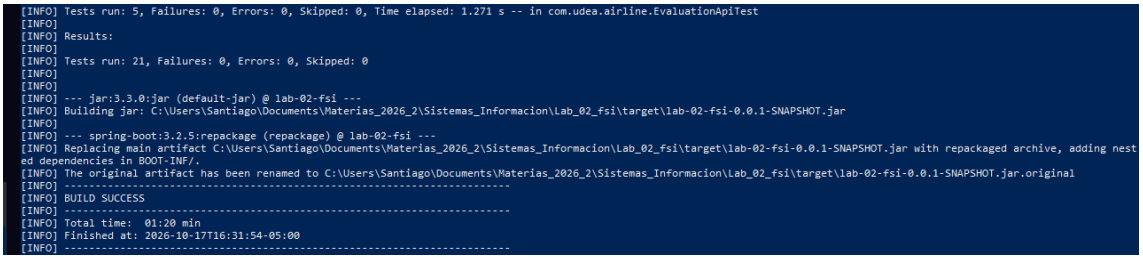


Ilustración 4 Primer resultado de compilación con 21 pruebas aprobadas. Fuente: elaboración propia a partir de las evidencias del laboratorio

Después de reorganizar el recurso DRL y ajustar el inventario de asientos, se incorporaron tres pruebas adicionales. La ejecución posterior reportó 24 pruebas aprobadas, cero fallidas, cero errores y cero omitidas, con BUILD SUCCESS. La advertencia inicial sobre ubicación del DRL desapareció. Durante las pruebas permaneció una advertencia de la JVM sobre class sharing, que no impidió la ejecución ni el empaquetado

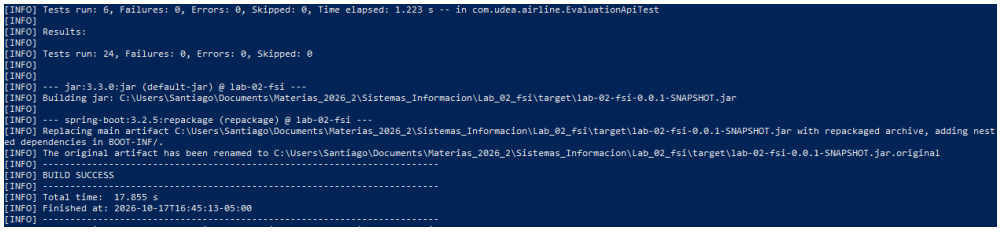


Ilustración 5. Resultados finales de Maven después de los ajustes. Fuente: elaboración propia a partir de las evidencias del laboratorio

4.7 Ejecución local de la aplicación

Una vez generado el JAR, se inició la aplicación con Java 17 y se verificó el arranque del servidor integrado Tomcat en el puerto 8080. La conectividad local fue comprobada mediante Test-NetConnection, obteniendo TcpTestSucceeded:

True. Esta comprobación confirma que existe un servicio escuchando en el puerto; las peticiones HTTP posteriores permiten validar específicamente el comportamiento de la API

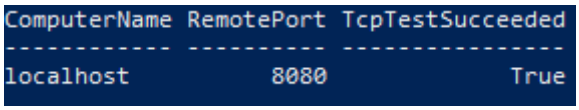


Ilustración 6. Inicio de Spring Boot y conectividad TCP en localhost:8080. Fuente: elaboración propia a partir de las evidencias del laboratorio

4.8 Evaluación HTTP de un pasajero

Se realizó una solicitud HTTP POST al endpoint /api/evaluations/passenger con el archivo de ejemplo examples/passenger.json. El caso identificado como P1 produjo una respuesta exitosa y permitió observar simultáneamente beneficios y restricciones. El resultado mostró ausencia de ascenso, no elegibilidad para ascenso, compensación de 200 unidades monetarias, 500 puntos adicionales, 600 puntos totales, acceso VIP y asiento E1 asignado.

El caso es útil para demostrar que un pasajero puede recibir varios beneficios sin que ello elimine una restricción específica, como el veto al ascenso por equipaje. La API devolvió HTTP 200 en la ejecución observada.

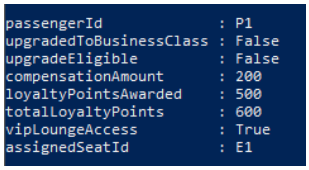


Ilustración 7. Respuesta a POST/api/evaluations/passenger con resultados del pasajero P1. Fuente: elaboración propia a partir de las evidencias del laboratorio

4.9 Evaluación de múltiples pasajeros y asiento escaso

Para estudiar la competencia por un recurso compartido, se ejecutó POST /api/evaluations/flight con dos pasajeros, FAMILIA e INDIVIDUAL, y únicamente un asiento disponible identificado como E1. El resultado HTTP 200 asignó E1 a FAMILIA; INDIVIDUAL quedó sin asiento, acompañado del mensaje «No hay asiento de salida de emergencia disponible». La consulta del estado final de los asientos permitió comprobar que el recurso se asignó una sola vez

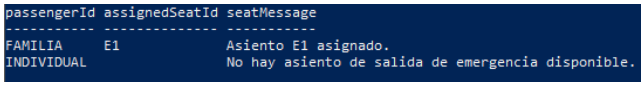


Ilustración 8. Prioridad familiar sin duplicación del asiento E1. Fuente: elaboración propia a partir de las evidencias del laboratorio

Adicionalmente, durante la verificación funcional se comprobaron los casos de frontera de los umbrales, la repetición de evaluaciones sin acumulación y cinco escenarios de entrada inválida rechazados con HTTP 400. Los registros textuales

se conservaron localmente como respaldo de la ejecución y no como parte del repositorio público

5. Conclusiones

El laboratorio permitió construir una solución que separa las políticas del negocio de los mecanismos de exposición HTTP. La definición de las diez reglas mediante DRL demuestra la utilidad del enfoque declarativo para mantener condiciones y consecuencias en un recurso específico, con menor acoplamiento respecto de la estructura de controladores y servicios.

La principal dificultad técnica no consistió solamente en implementar los umbrales individuales, sino en definir el comportamiento cuando varias reglas se activan sobre un mismo pasajero o compiten por recursos compartidos. El veto de ascenso por exceso de equipaje y la asignación única de asientos evidencian la necesidad de explicitar prioridades y mantener la coherencia del estado evaluado.

Los resultados de 24 pruebas automatizadas aprobadas y la ejecución de solicitudes HTTP reales aportaron evidencia de funcionamiento para los escenarios evaluados. En particular, se comprobaron reglas aisladas, combinaciones, límites estrictos, respuestas de validación y asignación de asientos. Esto no prueba la ausencia absoluta de defectos, pero sí constituye una validación reproducible de los requisitos principales del ejercicio.

La integración Spring Boot–Drools con Java 17 y Maven resultó operativa en el entorno utilizado. Como límite del alcance, el proyecto demuestra reglas de negocio en un escenario académico y no incorpora necesariamente persistencia, transacciones distribuidas ni mecanismos de reserva concurrente a escala real. Esas capacidades serían necesarias si se pretendiera evolucionar la solución hacia un sistema operativo de una aerolínea.

6. Bibliografía

[1] Universidad de Antioquia. (2026). Laboratorio Nro 2. Reglas del Negocio con Spring Boot y Drools. Guía de la asignatura Fundamentos de Sistemas de Información, pp. 1-30.

[2] Drools. (s. f.). Drools Documentation, versión 7.74.1.Final. https://docs.jboss.org/drools/release/7.74.1.Final/drools-docs/html_single/index.html

[3] Spring. (2024). Spring Boot 3.2.5 Reference Documentation: Getting Started. https://docs.spring.io/spring-boot/docs/3.2.5/reference/html/getting-started.html

[4] Apache Software Foundation. (s. f.). Introduction to the Build Lifecycle. Apache Maven Project. https://maven.apache.org/guides/introduction/introduction-to-the-lifecycle.html

7. Código de GitHub

Repositorio: Repositorio_Lab_02